Step-by-step Instructions for Assignment 3

February 8, 2026


Compilers
Principles, Techniques, and Tools
COMP2140

Outline

A2
Grammar
Run calc example
Change to A3.lex and A3.cup
Improve A3.lex
Remove evaluation
Expand A3
Turn on debug parse
Count methods
Deal with illegal token
Ambiguous grammar and Shift/re-

First, fully understand A2. If you didn't score full marks, continue to work on the test site.

Outline

A2

Grammar

Run calc example

Change to A3.lex and A3.cup

Improve A3.lex

Remove evaluation

Expand A3

Turn on debug parse

Count methods

Deal with illegal token

Ambiguous grammar and Shift/re-

Read and Understand Grammar

- You must fully understand the grammar and implement that grammar correctly so that your parser can cope with all situations.
- Test your program on our sample inputs
- Generate your own test cases (both correct and incorrect TINY programs).
- Passing the sample inputs will not guarantee a high mark.
- Whether empty block, i.e.,

```
BEGIN   END
```

  is a valid statement?
- You need to check with the grammar

```
Block -> BEGIN Statement+ END
```

Grammar

- Whether the following is a valid function declaration:

```
INT f( )  BEGIN ... END
```

- The grammar for the formal parameter is:

```
MethodDecl -> Type [MAIN] Id '(' FormalParams ')' Block
FormalParams -> [FormalParam ( ',' FormalParam )* ]
```

Program A3User invokes the parser:

```
import java.io.*;
class A3User {
  public static void main(String[] args) throws Exception {
        File inputFile = new File ("A3.tiny");
        A3Parser parser= new A3Parser(new A3Scanner(new
            FileInputStream(inputFile)));
        Integer result =(Integer)parser.parse().value;
        FileWriter fw=new FileWriter(new File("A3.output"));
        fw.write("Number of methods: "+ result.intValue());
        fw.close();
  }
}
```

- If your lex and cup files are correct,
  - all of those command and especially A3User will run smoothly without any error report,
  - A3.output file will be created which should consists of one line as follows:

```
Number of methods: numberOfMehtodsInA2Input
```

- If your programs are not correct, during the process there will be some error messages.

Outline

A2
Grammar
Run calc example
Change to A3.lex and A3.cup
Improve A3.lex
Remove evaluation
Expand A3
Turn on debug parse
Count methods
Deal with illegal token
Ambiguous grammar and Shift/re-

Run calc.lex and calc.cup example

Here is the list of commands to run:

```
java JLex.Main calc.lex
java java_cup.Main -parser CalcParser -symbols CalcSymbol calc.cup
javac calc.lex.java
javac CalcParser.java CalcSymbol.java CalcParserUser.java
java CalcParserUser
```

- Create a clean directory that contains your lex and cup files (possibly input files), and the JLex and java_cup directories.
  - For the installation of JLex and java_cup, refer the course web site.
- Run the commands line by line
- After each command, check what file(s) are generated using

  ```
  ls -lt
  ```

  which means to list the files sorted by time, the most recent ones at the top.

Generate the Scanner


```
>java JLex.Main calc.lex
```

If everything is right, you will see the following file(scanner) being generated:

```
>ls -lt
calc.lex.java
```

One common error at this stage

At this stage if you compile it using

```
javac calc.lex.java
```

- You will see a compilation error.
- It needs class CalcSymbol. See the calc.lex below.
- But CalcSymbol is not there yet.
- Check the files in the directory to confirm this.
- Therefore we need to generate the Parser and the symbol table (CalcSymbol).

```
import java_cup.runtime.*;
%%
%implements java_cup.runtime.Scanner
%type Symbol
%function next_token
%class CalcScanner
%eofval{ return null;
%eofval}
%%
"+" { return new Symbol(CalcSymbol.PLUS); }
"-" { return new Symbol(CalcSymbol.MINUS); }
"*" { return new Symbol(CalcSymbol.TIMES); }
"/" { return new Symbol(CalcSymbol.DIVIDE); }
...
```

Generate the parser

Here is the command to generate the parser:

```
java java_cup.Main -parser CalcParser -symbols CalcSymbol calc.cup
```

- You can check that CalcParser.java and CalSymbol.java are generated
- the parser name and symbol table name are given in the command line
- calc.cup is the input specification

Outline

A2
Grammar
Run calc example
Change to A3.lex and A3.cup
Improve A3.lex
Remove evaluation
Expand A3
Turn on debug parse
Count methods
Deal with illegal token
Ambiguous grammar and Shift/re-

Change the name to A3

- Change calc.lex and calc.cup to A3.lex and A3.cup.
- It is more than just changing file names
- You need to change every occurrence of **Calc** to **A3** in the lex, cup, and script file.
- Make sure that you can run A3.
- In this step, try to understand how A3.lex and A3.cup work together.

Change to A3 in the script


```
java JLex.Main A3.lex
java java_cup.Main -parser A3Parser -symbols A3Symbol A3.cup
javac A3.lex.java A3Parser.java A3Symbol.java A3User.java
java A3User
```

Compare with the calc example:

```
java JLex.Main calc.lex
java java_cup.Main -parser CalcParser -symbols CalcSymbol calc.cup
javac calc.lex.java
javac CalcParser.java CalcSymbol.java CalcParserUser.java
java CalcParserUser
```

Change to A3.lex

In the following calc.lex file, what else need to be changed?

```
%%
%implements java_cup.runtime.Scanner
%type Symbol
%function next_token
%class CalcScanner
%eofval{ return null;
%eofval}

IDENTIFIER = [a-zA-Z_][a-zA-Z0-9_]*
NUMBER = [0-9]+
%%
"+" { return new Symbol(CalcSymbol.PLUS); }
 ...
{NUMBER} { return new Symbol(CalcSymbol.NUMBER, new Integer(yytext
     ()));}
\r|\n|. {}
```

Change to A3.lex

In the following calc.lex file, what else need to be changed?

```
%%
%implements java_cup.runtime.Scanner
%type Symbol
%function next_token
%class CalcScanner
%eofval{ return null;
%eofval}

IDENTIFIER = [a-zA-Z_][a-zA-Z0-9_]*
NUMBER = [0-9]+
%%
"+" { return new Symbol(CalcSymbol.PLUS); }
 ...
{NUMBER} { return new Symbol(CalcSymbol.NUMBER, new Integer(yytext
     ()));}
\r|\n|. {}
```

- CalcScanner needs to be changed to A3Scanner,

Change to A3.lex

In the following calc.lex file, what else need to be changed?

```
%%
%implements java_cup.runtime.Scanner
%type Symbol
%function next_token
%class CalcScanner
%eofval{ return null;
%eofval}

IDENTIFIER = [a-zA-Z_][a-zA-Z0-9_]*
NUMBER = [0-9]+
%%
"+" { return new Symbol(CalcSymbol.PLUS); }
...
{NUMBER} { return new Symbol(CalcSymbol.NUMBER, new Integer(yytext
    ()));}
\r|\n|. {}
```

- CalcScanner needs to be changed to A3Scanner,
- CalcSymbol needs to be changed to A3Symbol.

Change to A3.lex

In the following calc.lex file, what else need to be changed?

```
%%
%implements java_cup.runtime.Scanner
%type Symbol
%function next_token
%class CalcScanner
%eofval{ return null;
%eofval}

IDENTIFIER = [a-zA-Z_][a-zA-Z0-9_]*
NUMBER = [0-9]+
%%
"+" { return new Symbol(CalcSymbol.PLUS); }
...
{NUMBER} { return new Symbol(CalcSymbol.NUMBER, new Integer(yytext
    ()));}
\r|\n|. {}
```

- CalcScanner needs to be changed to A3Scanner,
- CalcSymbol needs to be changed to A3Symbol.

Outline

A2
Grammar
Run calc example
Change to A3.lex and A3.cup
Improve A3.lex
Remove evaluation
Expand A3
Turn on debug parse
Count methods
Deal with illegal token
Ambiguous grammar and Shift/re-

Improve A3.lex according to A2

In the following calc.lex file, what need to be changed?

```
%%
%implements java_cup.runtime.Scanner
%type Symbol
%function next_token
%class CalcScanner
%eofval{ return null;
%eofval}

IDENTIFIER = [a-zA-Z_][a-zA-Z0-9_]*
NUMBER = [0-9]+
%%
"+" { return new Symbol(CalcSymbol.PLUS); }
 ...
{NUMBER} { return new Symbol(CalcSymbol.NUMBER, new Integer(yytext
      ()));}
\r|\n|. {}
```

Improve A3.lex according to A2

In the following calc.lex file, what need to be changed?

```
%%
%implements java_cup.runtime.Scanner
%type Symbol
%function next_token
%class CalcScanner
%eofval{ return null;
%eofval}

IDENTIFIER = [a-zA-Z_][a-zA-Z0-9_]*
NUMBER = [0-9]+
%%
"+" { return new Symbol(CalcSymbol.PLUS); }
...
{NUMBER} { return new Symbol(CalcSymbol.NUMBER, new Integer(yytext
    ()));}
\r|\n|. {}
```

- underscore is not part of an identifier

Improve A3.lex according to A2

In the following calc.lex file, what need to be changed?

```
%%
%implements java_cup.runtime.Scanner
%type Symbol
%function next_token
%class CalcScanner
%eofval{ return null;
%eofval}

IDENTIFIER = [a-zA-Z_][a-zA-Z0-9_]*
NUMBER = [0-9]+
%%
"+" { return new Symbol(CalcSymbol.PLUS); }
...
{NUMBER} { return new Symbol(CalcSymbol.NUMBER, new Integer(yytext
    ()));}
\r|\n|. {}
```

- underscore is not part of an identifier
- Add comment state

Improve A3.lex according to A2

In the following calc.lex file, what need to be changed?

```
%%
%implements java_cup.runtime.Scanner
%type Symbol
%function next_token
%class CalcScanner
%eofval{ return null;
%eofval}

IDENTIFIER = [a-zA-Z_][a-zA-Z0-9_]*
NUMBER = [0-9]+
%%
"+" { return new Symbol(CalcSymbol.PLUS); }
 ...
{NUMBER} { return new Symbol(CalcSymbol.NUMBER, new Integer(yytext
      ()));}
\r|\n|. {}
```

- underscore is not part of an identifier
- Add comment state
- Add more token definitions

Outline

Remove the evaluation function

This assignment won't evaluate the values of expressions, hence we should remove that functionality. To do so, we need to remove the RESULT assignment statements in the cup file:

```
terminal PLUS, MINUS, TIMES, DIVIDE, LPAREN, RPAREN;
terminal Integer  NUMBER;
non terminal Integer expr;
precedence left PLUS, MINUS;
precedence left TIMES, DIVIDE;
expr ::= expr:e1 PLUS expr:e2
    {: RESULT= e1 + e2; :}
    ...
    | LPAREN expr:e RPAREN {: RESULT = e; :}
    | NUMBER:e {: RESULT= e; :}
    ;
```

That is, your A3.cup file should look like this:

```
expr ::= expr PLUS expr {:  :}
   | expr MINUS expr  {: :}
....;
```

Note that labels like e1 and e2 are removed, along with the action code inside the brackets {: and :} .

- Make sure that you can run A3
- At this stage it can only check whether the input is a valid expression
- To check bigger syntactic entities, you need to extend the grammar
- Alway extend the grammar one rule at a time

Outline

Expand A3.cup file with one more rule

Expand A3 one rule at a time;

```
terminal PLUS, MINUS, TIMES, DIVIDE, LPAREN, RPAREN;
terminal Integer NUMBER;
non terminal Integer expr;
...
assignmentStatement::=ID EQ expr;
expr        ::= ...
;
```

- change your test case accordingly, i.e., your test case has one assignment state like

```
x=2+3;
```

- What else need to be changed ?

Expand A3.cup file with one more rule

Expand A3 one rule at a time;

```
terminal PLUS, MINUS, TIMES, DIVIDE, LPAREN, RPAREN;
terminal Integer NUMBER;
non terminal Integer expr;
...
assignmentStatement::=ID EQ expr;
expr        ::= ...
;
```

- change your test case accordingly, i.e., your test case has one assignment state like

  ```
  x=2+3;
  ```

- What else need to be changed ?
- Declare EQ as a terminal

Expand A3.cup file with one more rule

Expand A3 one rule at a time;

```
terminal PLUS, MINUS, TIMES, DIVIDE, LPAREN, RPAREN;
terminal Integer NUMBER;
non terminal Integer expr;
...
assignmentStatement::=ID EQ expr;
expr        ::= ...
;
```

- change your test case accordingly, i.e., your test case has one assignment state like

  ```
  x=2+3;
  ```

- What else need to be changed ?
- Declare EQ as a terminal
- Declare ID as a terminal

Expand A3.cup file with one more rule

Expand A3 one rule at a time;

```
terminal PLUS, MINUS, TIMES, DIVIDE, LPAREN, RPAREN;
terminal Integer NUMBER;
non terminal Integer expr;
...
assignmentStatement::=ID EQ expr;
expr        ::= ...
;
```

- change your test case accordingly, i.e., your test case has one assignment state like

  ```
  x=2+3;
  ```

- What else need to be changed ?
- Declare EQ as a terminal
- Declare ID as a terminal
- Make sure A3.lex returns ID and EQ

Remove Syntax errors in A3.cup and A3.lex


```
Parsing specification from standard input...
Error at 13(12): Syntax error
Error at 14(25): Illegal use of reserved word
Closing files...
-------- CUP v0.10k Parser Generation Summary --------
  2 errors and 0 warnings
  28 terminals, 26 non-terminals, and 47 productions declared,
  producing 0 unique parse states.
```

Here is the line 13 in the Cup file:

```
assignStmt    ::= ID ASSIGN expr
expr ::= ...
```

Remove Syntax errors in A3.cup and A3.lex


```
Parsing specification from standard input...
Error at 13(12): Syntax error
Error at 14(25): Illegal use of reserved word
Closing files...
-------- CUP v0.10k Parser Generation Summary --------
  2 errors and 0 warnings
  28 terminals, 26 non-terminals, and 47 productions declared,
  producing 0 unique parse states.
```

Here is the line 13 in the Cup file:

```
assignStmt    ::= ID ASSIGN expr
expr ::= ...
```

Frequent Error: Missing semicolon at the end of a rule

Outline

When the parser starts to work


```
Syntax error
Couldn't repair and continue parse
Exception in thread "main" java.lang.Exception: Can't recover from
    previous error(s)
at java_cup.runtime.lr_parser.report_fatal_error(lr_parser.java
    :361)
at java_cup.runtime.lr_parser.unrecovered_syntax_error(lr_parser.
    java:409)
at java_cup.runtime.lr_parser.parse(lr_parser.java:601)
at A3UserLu.main(A3UserLu.java:6)
```

- When you see this, congratulations!
- You have finished half of the work
- This means that the parser works!
- It is telling you that your input is incorrect
  - or, the grammar is incorrect
  - to be more precise, the input and the grammar is inconsistent
- how to proceed to debug?

Turn on debug parse

change parse() to debug_parse() in A3User:

```
class A3User {
  public static void main(String[] args) throws Exception {
      ...
       Integer result =(Integer)parser. debug_parse().value;
      ...
   }
```

Now you can see more debugging info


```
# Initializing parser
# Current Symbol is #11
Syntax error
# Attempting error recovery
# Finding recovery state on stack
# Pop stack by one, state was # 0
# No recovery state found on stack
# Error recovery fails
Couldn't repair and continue parse
```

- Current symbol is 11.
- it is the first token looked at by the parser in this case. Normally you could see a lot of **Current Symbol**s.
- The last **Current Symbol** is where the parser breaks.
- Symbols before the breaking point means that the grammar is ok for those input.
- You can look at the breaking point in the input to guess where it could be wrong.
- The error could be in the Scanner or the Parser (grammar).

Symbols are defined by A3Symbol

- In this case, when it sees the first symbol, the parser already knows that the input is invalid
- what is symbol 11?

```
//----------------------------------------------------
// The following code was generated by CUP v0.10k
// Tue Feb 26 11:56:56 EST 2019
//----------------------------------------------------

/** CUP generated class containing symbol constants. */
public class A3Symbol {
  /* terminals */
  public static final int TIMES = 22;
  public static final int READ = 17;
  public static final int ELSE = 4;
  public static final int PLUS = 20;
  public static final int RPAREN = 25;
 public static final int INT = 11;
  public static final int EQUAL = 5;
...
}
```

Mismatch between input and grammar

- Your input is

```
INT f() ....
```

- Your grammar is

```
ID EQ expr; ...
```

- There is a mismatch;
- You can change the input into an assignment statement to test your parser
- Once the first rule is tested, you can try to expand to other rules.

Parsing x:=2+3*5


```
# Initializing parser
# Current Symbol is #10
# Shift under term #10 to state #1
# Current token is #8
# Shift under term #8 to state #4
# Current token is #26
# Shift under term #26 to state #6
# Current token is #20
# Reduce with prod #44 [NT=6, SZ=1]
# Reduce rule: top state 4, lhs sym 6 -> state 7
# Goto state #7
# Shift under term #20 to state #11
# Current token is #26
# Shift under term #26 to state #6
# Current token is #22
# Reduce with prod #44 [NT=6, SZ=1]
# Reduce rule: top state 11, lhs sym 6 -> state 18
# Goto state #18
# Shift under term #22 to state #14
# Current token is #26
# Shift under term #26 to state #6
# Current token is #0
# Reduce with prod #44 [NT=6, SZ=1]
# Reduce rule: top state 14, lhs sym 6 -> state 15
# Goto state #15
# Reduce with prod #41 [NT=6, SZ=3]
# Reduce rule: top state 11, lhs sym 6 -> state 18
# Goto state #18
# Reduce with prod #39 [NT=6, SZ=3]
```

```
    public static final int TIMES = 22;
    public static final int READ = 17;
    public static final int ELSE = 4;
    public static final int PLUS = 20;
    public static final int RPAREN = 25;
    public static final int INT = 11;
    public static final int EQUAL = 5;
    public static final int THEN = 3;
    public static final int SEMI = 16;
    public static final int NOTEQUAL = 6;
    public static final int RETURN = 19;
    public static final int END = 15;
    public static final int IF = 2;
    public static final int LPAREN = 24;
    public static final int WRITE = 18;
    public static final int ID = 10;
    public static final int BEGIN = 14;
    public static final int COMA = 9;
    public static final int NUMBER = 26;
    public static final int EOF = 0;
    public static final int DIVIDE = 23;
    public static final int MAIN = 7;
    public static final int m = 27;
    public static final int MINUS = 21;
    public static final int error = 1;
    public static final int ASSIGN = 8;
    public static final int QSTRING = 13;
    public static final int REAL = 12;
}
```

Outline

Counting

The top level rule:

```
pm::= pm m
    | m
    ;
```

Action code added:

```
pm::= pm:e m {: RESULT=1+e; :}
    | m {: RESULT=1; :}
    ;
```

You do not need to have action code for other rules

Outline

Illegal symbol problem

The following program is incorrect because of symbols @ and !.

```
INT MAIN f1()
@!
BEGIN
    REAL x;
END
```

Your parser may treat that as a correct program because your scanner is defined as below:

```
...
\r|\n {}
. {}
```

The last line says that when we see 'other' symbols, such as '@', we throw it away.

How to correct it


```
INT MAIN f1()  @!
BEGIN
    REAL x;
END
```

- We can not throw all other symbols away
- We need to return a token that is unexpected, or, that is an error token.
- At the same time we want to make sure that all space tokens are discarded. They include spaces, tabs, new lines.
- hence those two lines should be changed into below:
- A3Symbol.error is the default error token defined by the tool.

```
...
\r|\n|\t|" " {}
. { return new Symbol(A3Symbol.error); }
```

The last line says that when we see 'other' symbols, such as '@', we throw it away.

Outline

A2
Grammar
Run calc example
Change to A3.lex and A3.cup
Improve A3.lex
Remove evaluation
Expand A3
Turn on debug parse
Count methods
Deal with illegal token
Ambiguous grammar and Shift/re-

Shift/reduce error

- Shift/reduce error: Most probably it is because that your grammar is ambiguous.
- You can either remove the ambiguity as we discussed in class, or add `precedence` rules.
- The simplest way to remove the ambiguity is to remove redundant rules.

Outline

Shorter code

- Group multiple operators as one operator in your lex file.
  - This can be applied to arithmetic expressions and boolean expressions.
  - This time our parser will not evaluate the program– it only needs to tell whether the code is syntactically valid.
  - Hence we do not need to tell the difference of operations. For instance, it does not matter whether it is "+" or "-".
- Factor out the common part:
  - if one entity is used in many places, you can factor them out.
  - To draw an analogy in arithmetics: instead of writing 1*2+2*2+3*2+4*2+5*2, you can make it shorter by move out the common factor 2 by rewriting it into (1+2+3+4+5)*2.

Outline

A2
Grammar
Run calc example
Change to A3.lex and A3.cup
Improve A3.lex
Remove evaluation
Expand A3
Turn on debug parse
Count methods
Deal with illegal token
Ambiguous grammar and Shift/re-

Test cases

test case	Problem tested
1	need to return error token for symbols such as @
2	read statement
3	Block statement can not be empty
4	Underscore is not allowed
5	write statement can take an expression. Real number
6	more than 2 methods
7	long expression, IF stmt
8	IF stmt
9	Expression, (x+2)
10	expression
11	long expression, long program
12	semi colon